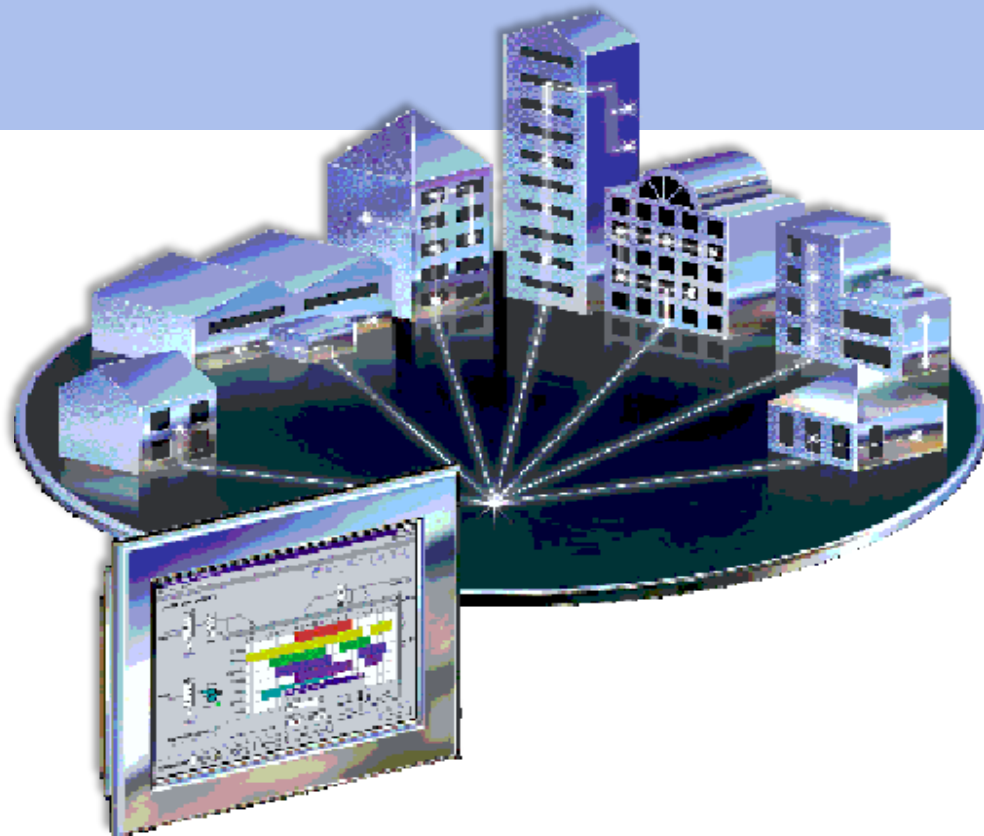


INSTITUTO POLITÉCNICO NACIONAL
ESCUELA SUPERIOR DE CÓMPUTO



Práctica 3:

MULTI-CLIENTE SERVIDOR



SISTEMAS DISTRIBUIDOS

Profesor || Chadwick Carreto Arellano

Grupo || 7CM1

Alumno || Areli Alejandra Guevara Badillo

Febrero 27, 2026

ANTECEDENTES

En el contexto de los sistemas distribuidos y la programación de redes, la transición de un modelo un cliente-un servidor a un modelo múltiples clientes-un servidor introduce complejidades significativas relacionadas con la concurrencia y la integridad de los datos.

Cuando un servidor atiende a múltiples clientes simultáneamente, generalmente lo hace mediante el uso de hilos de ejecución (threads) o procesos ligeros, permitiendo que cada conexión sea manejada de manera independiente sin bloquear la aceptación de nuevas conexiones.

MECANISMOS DE SINCRONIZACIÓN Y CONCURRENCIA

La ejecución concurrente de múltiples hilos que acceden a recursos compartidos (como variables globales, archivos o estructuras de datos en memoria) puede dar lugar a condiciones de carrera (race conditions). Esto ocurre cuando el comportamiento del sistema depende del orden no determinista en que se ejecutan los hilos, pudiendo resultar en corrupción de datos o inconsistencias. Para mitigar esto, se emplean mecanismos de sincronización y concurrencia.

Los mecanismos de sincronización y concurrencia gestionan el acceso de múltiples hilos o procesos a recursos compartidos, garantizando la consistencia de los datos y evitando conflictos. Utilizan técnicas como:

- **Sección Crítica:** Fragmento de código donde se accede a recursos compartidos y que no debería ser ejecutado por más de un hilo a la vez.
- **Exclusión Mutua (Lock/Mutex):** Primitive de sincronización que permite que solo un hilo adquiera el "candado" para entrar a la sección crítica. En Python, se implementa mediante `threading.Lock()`.
- **Semáforos:** Contadores generalizados que permiten limitar el número de hilos que pueden acceder a un recurso simultáneamente. Son útiles para controlar la carga del servidor (ej. `threading.Semaphore()`).
- **Variables de Condición:** Permiten a los hilos esperar hasta que se cumpla una cierta condición antes de proceder, facilitando la comunicación entre hilos (`threading.Condition()`).
- **Monitores:** Construcciones de lenguaje de alto nivel que encapsulan variables y procedimientos, garantizando que solo un hilo ejecute una función del monitor a la vez.

La implementación correcta de estos esquemas es vital para garantizar la consistencia eventual y la disponibilidad del servicio en entornos de alta demanda, asegurando que las operaciones de lectura y escritura sobre el almacenamiento compartido sean atómicas y seguras.

PLANTEAMIENTO DEL PROBLEMA

En la era actual de computación en la nube y servicios remotos, existe una tensión constante entre la accesibilidad de los datos y la privacidad del usuario. El objetivo principal de este proyecto es construir un sistema de **Almacenamiento de Secretos con Conocimiento Cero (Zero-Knowledge Storage)**.

En el escenario planteado, un Cliente necesita guardar información sensible (contraseñas, claves privadas, notas confidenciales) en un Servidor remoto para asegurar su disponibilidad. Sin embargo, el requisito criptográfico fundamental es que el Servidor nunca debe poder leer la información que almacena. El servidor debe actuar simplemente como un depósito de datos cifrados ("caja ciega"), sin poseer las capacidades ni las claves para descifrar el contenido.

El problema se complejiza al requerir que el sistema soporte **múltiples clientes concurrentes**. Es necesario que, ante solicitudes simultáneas de escritura o lectura sobre los mismos identificadores de secretos, no se produzcan condiciones de carrera que corrompan el almacenamiento compartido, manteniendo la seguridad criptográfica y la responsividad del sistema.

PROPUESTA DE SOLUCIÓN

Se propone una arquitectura Cliente-Servidor asíncrona basada en sockets TCP, bajo el principio de Privacidad Zero-Knowledge, ampliada con mecanismos de control de concurrencia.

Nodo Cliente: Responsable de la interfaz de usuario (UI), la gestión de claves maestras y las operaciones criptográficas. El cifrado ocurre antes de la transmisión de red. Se implementan bloqueos locales para evitar solicitudes superpuestas desde la misma interfaz.

Nodo Servidor: Actúa como un repositorio ciego. Recibe datos cifrados, los asocia a un identificador único (`secret_id`) y los almacena. No posee las claves de descifrado. Incorpora primitivas de sincronización para gestión de acceso concurrente al diccionario de almacenamiento.

Modelo de Seguridad y Control

- **Cifrado Simétrico Autenticado:** Se utiliza el esquema Fernet (AES-128-CBC con HMAC) mediante la librería `cryptography`. Esto garantiza confidencialidad e integridad.
- **Gestión de Claves:** La clave de cifrado se genera y reside exclusivamente en el cliente. Nunca se transmite por la red.
- **Sincronización de Recursos:** Se emplean Locks para proteger la estructura de datos compartida (`vault_storage`) y Semáforos para limitar la carga máxima de conexiones simultáneas.
- **Codificación:** Los datos cifrados (binarios) se codifican en Base64 para su transmisión segura dentro de paquetes JSON.

DESARROLLO E IMPLEMENTACIÓN

El desarrollo del sistema se llevó a cabo utilizando **Python 3.14** como lenguaje principal, aprovechando su ecosistema robusto para operaciones de red, criptografía, interfaces gráficas y manejo de concurrencia.

Para esta fase, se realizaron modificaciones significativas sobre la base de la Práctica 2, integrando módulos de las librerías `threading` y `collections` para manejar la concurrencia segura en un entorno multi-cliente. A continuación, se detalla la arquitectura técnica, las decisiones de diseño y la lógica interna de cada componente con sus respectivos fragmentos de código.

ENTORNO Y LIBRERÍAS

Para garantizar la seguridad, funcionalidad y concurrencia del sistema multi-cliente, se seleccionaron las siguientes librerías estándar y de terceros:

Librería	Propósito
<code>socket</code>	Comunicación de bajo nivel mediante TCP/IP
<code>threading</code>	Manejo de hilos para concurrencia en servidor y cliente
<code>json</code>	Serialización y deserialización de paquetes de datos
<code>cryptography.fernet</code>	Cifrado simétrico autenticado de alto nivel
<code>tkinter</code>	Construcción de la interfaz gráfica de usuario (GUI)
<code>base64</code>	Codificación de datos binarios cifrados en formato texto ASCII
<code>collections.deque</code>	Cola circular para registro de auditoría de operaciones
<code>time</code> y <code>datetime</code>	Manejo de timestamps para ordenamiento de operaciones

ARQUITECTURA DEL SERVIDOR (SERVIDOR.PY)

El servidor ahora atiende múltiples solicitudes simultáneas mediante hilos (`threading.Thread`). Para evitar condiciones de carrera sobre el diccionario compartido `vault_storage`, se implementaron primitivas de sincronización:

- **Exclusión Mutua (Lock):** Protege las secciones críticas donde se lee o escribe en el almacenamiento.
- **Semáforo (Semaphore):** Limita la cantidad máxima de conexiones concurrentes para evitar saturación.

Fragmento 1: Inicialización de Mecanismos de Control

```
# Lock para proteger el acceso al diccionario compartido
storage_lock = threading.Lock()
# Semáforo para limitar conexiones concurrentes
connection_semaphore = threading.Semaphore(10)
```

Fragmento 2: Sección Crítica Protegida

Al recibir una solicitud STORE o RETRIEVE, el hilo adquiere el lock antes de tocar los datos:

```
if action == "STORE":
    with storage_lock: # Entrada a sección crítica
        if secret_id and payload:
            vault_storage[secret_id] = payload
            # ... registro de auditoría ...
```

Fragmento 3: Control de Admisión

Cada hilo adquiere el semáforo al iniciar y lo libera al finalizar, asegurando el conteo correcto de conexiones activas:

```
def handle_client(conn, addr):
    connection_semaphore.acquire() # Bloquea si se alcanza el límite
    try:
        # ... lógica de atención ...
    finally:
        connection_semaphore.release() # Libera cupo
```

CLIENTE: ROBUSTEZ Y RESPONSABILIDAD

El cliente mantiene la interfaz gráfica en tkinter sin congelamientos, ejecutando operaciones de red en hilos secundarios. Se agregaron mecanismos para manejar la latencia y evitar conflictos locales:

- **Timeout de Socket:** Previene bloqueos indefinidos si el servidor no responde.
- **Lock Local:** Serializa las solicitudes enviadas desde la misma interfaz.

Fragmento 4: Configuración de Timeout y Lock Local

```
class CryptoVaultClient:
    def __init__(self, root):
        # ...
        self.operation_lock = threading.Lock() # Control Local

    def connect_to_server(self):
        self.sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        self.sock.settimeout(10) # Timeout de 10 segundos
        self.sock.connect((SERVER_HOST, SERVER_PORT))
```

Fragmento 5: Envío de Solicitudes Serializadas

```
def send_request(self, data_dict):
    with self.operation_lock: # Evita envío simultáneo desde la UI
        msg = json.dumps(data_dict).encode('utf-8')
        # ... envío y recepción ...
```

DISEÑO DEL PROTOCOLO DE COMUNICACIÓN

El protocolo JSON se extendió para incluir metadatos de control que facilitan la auditoría y el ordenamiento de operaciones:

```
# Estructura de solicitud extendida
{
  "action": "STORE",      # STORE, RETRIEVE, LIST, LOG, DISCONNECT
  "id": "secret_id",     # Identificador único del secreto
  "payload": "base64...", # Datos cifrados (solo para STORE)
  "timestamp": 1234567.89 # Timestamp Unix para ordenamiento
}
# Estructura de respuesta extendida
{
  "status": "success",   # success o error
  "message": "...",      # Mensaje descriptivo
  "payload": "base64...", # Datos cifrados (solo para RETRIEVE)
  "operation": "CREATE",  # CREATE o UPDATE (solo para STORE)
  "ids": [...],          # Lista de IDs (solo para LIST)
  "log": [...]           # Registro de operaciones (solo para LOG)
}
```

FLUJO CRIPTOGRÁFICO DETALLADO

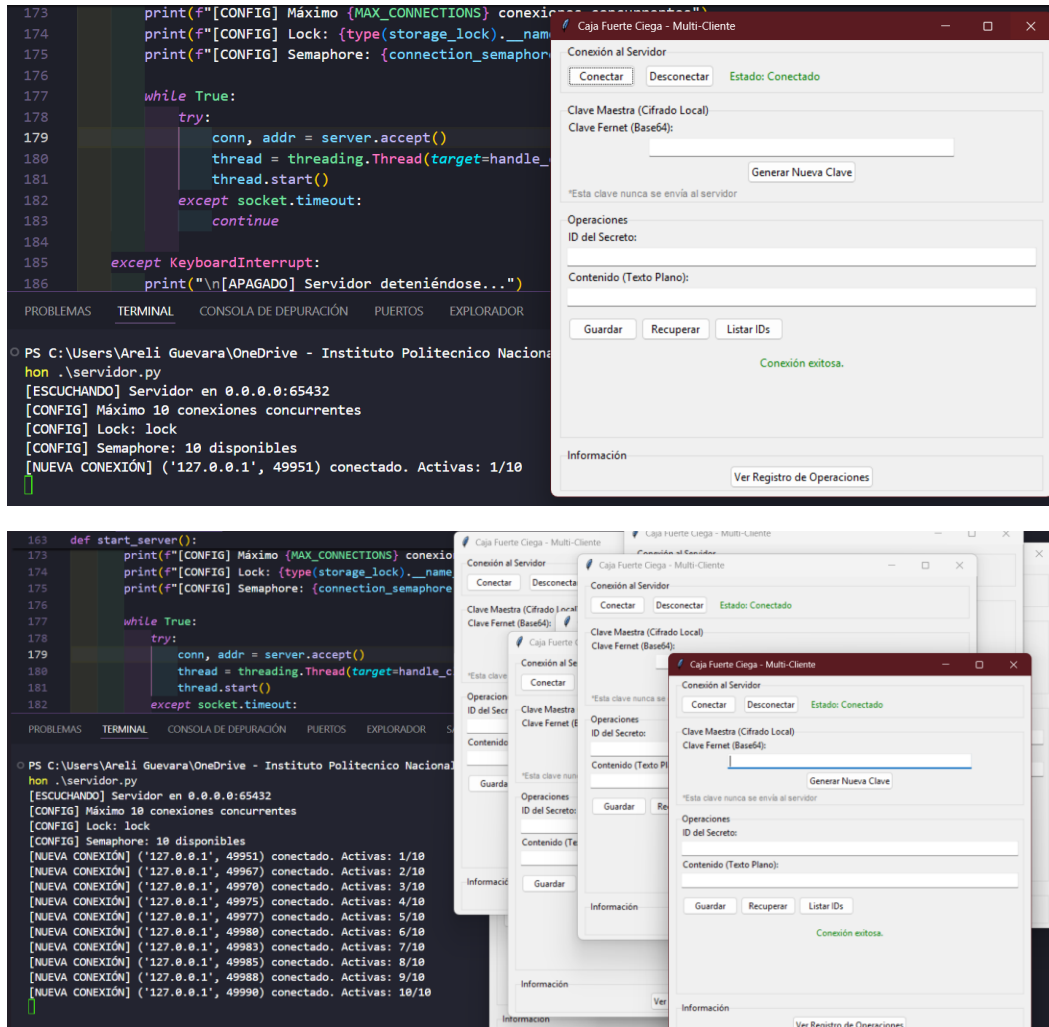
La seguridad del sistema reside en el proceso de cifrado/descifrado que ocurre exclusivamente en el cliente, manteniendo al servidor en ceguera respecto al contenido. Este flujo se mantiene igual que en la Práctica 2, pero ahora opera en un entorno concurrente:

```
# Cifrado en el cliente (antes de enviar)
def store_secret(self):
    key = self.entry_key.get().encode('utf-8')
    f = Fernet(key)
    content = self.entry_content.get().encode('utf-8')
    # Cifrado Local Zero-Knowledge
    encrypted_token = f.encrypt(content)
    payload_b64 = base64.b64encode(encrypted_token).decode('utf-8')
    # Envío al servidor (el servidor nunca ve el texto plano)
    response = self.send_request({
        "action": "STORE",
        "id": secret_id,
        "payload": payload_b64,
        "timestamp": time.time()
    })

# Descifrado en el cliente (después de recibir)
def retrieve_secret(self):
    key = self.entry_key.get().encode('utf-8')
    f = Fernet(key)
    response = self.send_request({
        "action": "RETRIEVE",
        "id": secret_id
    })
    if response['status'] == 'success':
        payload_b64 = response['payload']
        encrypted_token = base64.b64decode(payload_b64)
        # Descifrado Local con la clave del cliente
        decrypted_content = f.decrypt(encrypted_token).decode('utf-8')
```

RESULTADOS

Las pruebas realizadas validaron el funcionamiento del sistema multi-cliente con mecanismos de sincronización y concurrencia. Se verificó la correcta implementación del protocolo de comunicación, la seguridad criptográfica y el control de acceso concurrente a recursos compartidos.



Se conectaron exitosamente múltiples instancias del cliente (3-5 simultáneas) al mismo servidor. El mecanismo de Lock protegió las operaciones de escritura sobre el almacenamiento compartido, evitando condiciones de carrera. El semáforo limitó correctamente las conexiones máximas configuradas (10), y el registro de auditoría mostró el orden cronológico de todas las operaciones realizadas por los diferentes clientes.

CONCLUSIONES

Mediante el desarrollo de la presente práctica de Concurrency y Sincronización en Multi-Cliente, se logró evolucionar el sistema de almacenamiento Zero-Knowledge hacia un modelo concurrente robusto. Se consolidó el entendimiento sobre la importancia de la sincronización en sistemas distribuidos, donde el acceso no controlado a recursos compartidos puede comprometer la consistencia de los datos incluso si la criptografía es segura.

La implementación de Locks y Semáforos en Python demostró ser efectiva para gestionar secciones críticas y limitar la carga del servidor, respectivamente. El mecanismo de exclusión mutua (`threading.Lock()`) protegió el diccionario compartido de condiciones de carrera, mientras que el semáforo (`threading.Semaphore()`) controló adecuadamente el número máximo de conexiones simultáneas. Además, la adición de mecanismos de auditoría con timestamps y timeouts mejoró la observabilidad y la resiliencia del sistema frente a fallos de red o saturación.

La arquitectura multi-cliente mantuvo intacto el principio de Privacidad Zero-Knowledge, demostrando que la seguridad criptográfica puede coexistir con la concurrencia operativa. Cada cliente continuó gestionando sus claves de forma local, mientras el servidor atendió múltiples solicitudes sin acceder al contenido sensible de los datos almacenados.

ANEXOS

A. Código del Servidor (servidor.py)

```
import socket
import threading
import json
import sys
import time
from datetime import datetime
from collections import deque

# Configuración del Servidor
HOST = '0.0.0.0'
PORT = 65432
MAX_CONNECTIONS = 10 # Límite de clientes concurrentes

# Almacenamiento compartido protegido
vault_storage = {}

# === MECANISMOS DE SINCRONIZACIÓN ===

# Lock para proteger el acceso al diccionario compartido
storage_lock = threading.Lock()

# Semaphore para limitar conexiones concurrentes
connection_semaphore = threading.Semaphore(MAX_CONNECTIONS)

# Condition para notificar cambios en el almacenamiento
storage_condition = threading.Condition()

# Cola para registrar operaciones (auditoría)
operation_log = deque(maxlen=100)
log_lock = threading.Lock()

# Contador de conexiones activas
active_connections = 0
connections_lock = threading.Lock()

def log_operation(operation_type, secret_id, addr, status):
    """Registra operaciones en la cola de auditoría con timestamp"""
    timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    with log_lock:
        operation_log.append({
            "timestamp": timestamp,
            "operation": operation_type,
```

```

        "id": secret_id,
        "client": str(addr),
        "status": status
    })

def handle_client(conn, addr):
    """
    Maneja la comunicación con un cliente específico.
    Incluye sincronización para acceso concurrente seguro.
    """
    global active_connections

    # Adquirir semáforo de conexión
    connection_semaphore.acquire()

    with connections_lock:
        active_connections += 1
        print(f"[NUEVA CONEXIÓN] {addr} conectado. Activas:
{active_connections}/{MAX_CONNECTIONS}")

    connected = True

    try:
        while connected:
            try:
                # Recibir longitud del mensaje
                msg_length = conn.recv(4).decode('utf-8')
                if not msg_length:
                    break
                msg_length = int(msg_length)

                # Recibir payload JSON
                message = conn.recv(msg_length).decode('utf-8')
                data = json.loads(message)

                action = data.get("action")
                secret_id = data.get("id")
                payload = data.get("payload")
                timestamp = data.get("timestamp", time.time())

                response = {"status": "error", "message": "Acción desconocida"}

                # === SECCIÓN CRÍTICA PROTEGIDA POR LOCK ===
                if action == "STORE":
                    with storage_lock: # Lock adquirido
                        if secret_id and payload:

```

```

        is_update = secret_id in vault_storage
        vault_storage[secret_id] = payload

        # Notificar a otras threads que hay cambio
        with storage_condition:
            storage_condition.notify_all()

        response = {
            "status": "success",
            "message": "Secreto almacenado (Cifrado)",
            "operation": "UPDATE" if is_update else "CREATE"
        }
        log_operation("STORE", secret_id, addr, "SUCCESS")
        print(f"[ALMACENAMIENTO] ID: {secret_id} por {addr}"
              f'{"(Actualizado)" if is_update else "(Nuevo)"}')
    else:
        response = {"status": "error", "message": "Faltan ID o
Payload"}

        log_operation("STORE", secret_id, addr, "FAILED")

elif action == "RETRIEVE":
    with storage_lock: # Lock adquirido
        if secret_id in vault_storage:
            response = {
                "status": "success",
                "message": "Secreto recuperado",
                "payload": vault_storage[secret_id]
            }
            log_operation("RETRIEVE", secret_id, addr, "SUCCESS")
            print(f"[RECUPERACIÓN] ID: {secret_id} enviado a {addr}")
        else:
            response = {"status": "error", "message": "ID no
encontrado"}

            log_operation("RETRIEVE", secret_id, addr, "FAILED")

elif action == "LIST":
    with storage_lock:
        response = {
            "status": "success",
            "message": "Lista de IDs disponibles",
            "ids": list(vault_storage.keys())
        }
        log_operation("LIST", "-", addr, "SUCCESS")

elif action == "LOG":
    with log_lock:

```

```

        response = {
            "status": "success",
            "message": "Registro de operaciones",
            "log": list(operation_log)
        }

    elif action == "DISCONNECT":
        connected = False
        response = {"status": "success", "message": "Desconectando"}

    # Enviar respuesta
    response_msg = json.dumps(response).encode('utf-8')
    msg_length = len(response_msg)
    conn.send(str(msg_length).encode('utf-8').ljust(4))
    conn.send(response_msg)

    except json.JSONDecodeError:
        print(f"[ERROR JSON] Datos inválidos de {addr}")
        response = {"status": "error", "message": "Formato JSON inválido"}
    except Exception as e:
        print(f"[ERROR] con {addr}: {e}")
        connected = False

finally:
    # Liberar recursos
    conn.close()
    connection_semaphore.release()

    with connections_lock:
        active_connections -= 1
        print(f"[CONEXIÓN CERRADA] {addr}. Activas: {active_connections}/{MAX_CONNECTIONS}")

def start_server():
    """Inicia el servidor con control de concurrencia"""
    server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    server.settimeout(1.0)

    try:
        server.bind((HOST, PORT))
        server.listen(MAX_CONNECTIONS)
        print(f"[ESCUCHANDO] Servidor en {HOST}:{PORT}")
        print(f"[CONFIG] Máximo {MAX_CONNECTIONS} conexiones concurrentes")
        print(f"[CONFIG] Lock: {type(storage_lock).__name__}")
        print(f"[CONFIG] Semaphore: {connection_semaphore._value} disponibles")
    
```

```
    while True:
        try:
            conn, addr = server.accept()
            thread = threading.Thread(target=handle_client, args=(conn, addr),
daemon=True)
            thread.start()
        except socket.timeout:
            continue

    except KeyboardInterrupt:
        print("\n[APAGADO] Servidor deteniéndose...")
        print(f"[ESTADÍSTICAS] Operaciones registradas: {len(operation_log)}")
        server.close()
        sys.exit()

if __name__ == "__main__":
    start_server()
```

B. Código del Cliente (cliente.py)

```
import socket
import threading
import json
import tkinter as tk
from tkinter import ttk, messagebox
from cryptography.fernet import Fernet
import base64
import time

# Configuración del Cliente
SERVER_HOST = '127.0.0.1'
SERVER_PORT = 65432

class CryptoVaultClient:
    def __init__(self, root):
        self.root = root
        self.root.title("Caja Fuerte Ciega - Multi-Cliente")
        self.root.geometry("550x500")

        self.sock = None
        self.connected = False
        self.operation_lock = threading.Lock() # Lock local para operaciones del
cliente

        self.create_widgets()

    def create_widgets(self):
        # Frame de Conexión
        conn_frame = ttk.LabelFrame(self.root, text="Conexión al Servidor")
        conn_frame.pack(fill="x", padx=10, pady=5)

        ttk.Button(conn_frame, text="Conectar",
command=self.connect_to_server).pack(side="left", padx=5, pady=5)
        ttk.Button(conn_frame, text="Desconectar",
command=self.disconnect).pack(side="left", padx=5, pady=5)
        self.lbl_status = ttk.Label(conn_frame, text="Estado: Desconectado",
foreground="red")
        self.lbl_status.pack(side="left", padx=10)

        # Frame de Clave Criptográfica
        key_frame = ttk.LabelFrame(self.root, text="Clave Maestra (Cifrado Local)")
        key_frame.pack(fill="x", padx=10, pady=5)

        ttk.Label(key_frame, text="Clave Fernet (Base64):").pack(anchor="w", padx=5)
```

Práctica 3: Multi-Cliente servidor

```
self.entry_key = ttk.Entry(key_frame, width=55)
self.entry_key.pack(padx=5, pady=2)

ttk.Button(key_frame, text="Generar Nueva Clave",
command=self.generate_key).pack(pady=2)
ttk.Label(key_frame, text="*Esta clave nunca se envía al servidor",
foreground="gray", font=("Arial", 8)).pack(anchor="w", padx=5)

# Frame de Operaciones
op_frame = ttk.LabelFrame(self.root, text="Operaciones")
op_frame.pack(fill="both", expand=True, padx=10, pady=5)

ttk.Label(op_frame, text="ID del Secreto:").pack(anchor="w", padx=5)
self.entry_id = ttk.Entry(op_frame)
self.entry_id.pack(fill="x", padx=5, pady=2)

ttk.Label(op_frame, text="Contenido (Texto Plano):").pack(anchor="w", padx=5)
self.entry_content = ttk.Entry(op_frame)
self.entry_content.pack(fill="x", padx=5, pady=2)

btn_frame = ttk.Frame(op_frame)
btn_frame.pack(fill="x", padx=5, pady=10)

ttk.Button(btn_frame, text="Guardar", command=lambda:
self.thread_action(self.store_secret)).pack(side="left", padx=2)
ttk.Button(btn_frame, text="Recuperar", command=lambda:
self.thread_action(self.retrieve_secret)).pack(side="left", padx=2)
ttk.Button(btn_frame, text="Listar IDs", command=lambda:
self.thread_action(self.list_ids)).pack(side="left", padx=2)

self.lbl_result = ttk.Label(op_frame, text="Resultado: ", wraplength=480,
foreground="blue")
self.lbl_result.pack(padx=5, pady=5)

# Frame de Información del Servidor
info_frame = ttk.LabelFrame(self.root, text="Información")
info_frame.pack(fill="x", padx=10, pady=5)

ttk.Button(info_frame, text="Ver Registro de Operaciones", command=lambda:
self.thread_action(self.get_log)).pack(pady=2)

def generate_key(self):
    key = Fernet.generate_key()
    self.entry_key.delete(0, tk.END)
    self.entry_key.insert(0, key.decode('utf-8'))
    messagebox.showinfo("Clave Generada", "Guarda esta clave en un lugar seguro.")
```

```

def connect_to_server(self):
    try:
        self.sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        self.sock.settimeout(10) # Timeout para operaciones
        self.sock.connect((SERVER_HOST, SERVER_PORT))
        self.connected = True
        self.lbl_status.config(text="Estado: Conectado", foreground="green")
        self.lbl_result.config(text="Conexión exitosa.", foreground="green")
    except Exception as e:
        messagebox.showerror("Error de Conexión", str(e))
        self.connected = False
        self.lbl_status.config(text="Estado: Error", foreground="red")

def disconnect(self):
    if self.connected and self.sock:
        try:
            self.send_request({"action": "DISCONNECT"})
            self.sock.close()
        except:
            pass
        finally:
            self.connected = False
            self.sock = None
            self.lbl_status.config(text="Estado: Desconectado", foreground="red")
            self.lbl_result.config(text="Desconectado del servidor.",
foreground="gray")

def send_request(self, data_dict):
    """Envía datos con control de concurrencia local"""
    if not self.connected or not self.sock:
        raise Exception("No hay conexión con el servidor")

    with self.operation_lock: # Evita operaciones simultáneas desde el mismo
cliente
        msg = json.dumps(data_dict).encode('utf-8')
        msg_length = len(msg)
        self.sock.send(str(msg_length).encode('utf-8').ljust(4))
        self.sock.send(msg)

        length_msg = self.sock.recv(4).decode('utf-8')
        if not length_msg:
            raise Exception("Conexión cerrada por el servidor")
        length = int(length_msg)
        response = self.sock.recv(length).decode('utf-8')
        return json.loads(response)

```



```

def store_secret(self):
    """Almacenamiento con validación y manejo de errores"""
    try:
        key = self.entry_key.get().encode('utf-8')
        if not key:
            raise ValueError("Se requiere clave maestra")

        f = Fernet(key)
        content = self.entry_content.get().encode('utf-8')
        secret_id = self.entry_id.get()

        if not content or not secret_id:
            raise ValueError("ID y Contenido obligatorios")

        encrypted_token = f.encrypt(content)
        payload_b64 = base64.b64encode(encrypted_token).decode('utf-8')

        response = self.send_request({
            "action": "STORE",
            "id": secret_id,
            "payload": payload_b64,
            "timestamp": time.time()
        })

        op_type = response.get("operation", "")
        self.lbl_result.config(
            text=f"{response['message']} {op_type}",
            foreground="green"
        )

        if response['status'] == 'success':
            self.entry_content.delete(0, tk.END)

    except Exception as e:
        self.lbl_result.config(text=f"Error: {str(e)}", foreground="red")

def retrieve_secret(self):
    """Recuperación con validación de clave"""
    try:
        key = self.entry_key.get().encode('utf-8')
        if not key:
            raise ValueError("Se requiere clave maestra")

        f = Fernet(key)
        secret_id = self.entry_id.get()

```

```

        if not secret_id:
            raise ValueError("ID obligatorio")

        response = self.send_request({
            "action": "RETRIEVE",
            "id": secret_id
        })

        if response['status'] == 'success':
            payload_b64 = response['payload']
            encrypted_token = base64.b64decode(payload_b64)
            decrypted_content = f.decrypt(encrypted_token).decode('utf-8')
            self.lbl_result.config(text=f"Secreto: {decrypted_content}",
foreground="green")
        else:
            self.lbl_result.config(text=f"Error: {response['message']}",
foreground="red")

    except Exception as e:
        self.lbl_result.config(text=f"Error: {str(e)}", foreground="red")

def list_ids(self):
    """Lista todos los IDs disponibles en el servidor"""
    try:
        response = self.send_request({"action": "LIST"})
        if response['status'] == 'success':
            ids = response.get("ids", [])
            self.lbl_result.config(text=f"IDs disponibles ({len(ids)}): {'',
'.join(ids)}", foreground="blue")
        else:
            self.lbl_result.config(text=f"Error: {response['message']}",
foreground="red")
    except Exception as e:
        self.lbl_result.config(text=f"Error: {str(e)}", foreground="red")

def get_log(self):
    """Obtiene el registro de operaciones del servidor"""
    try:
        response = self.send_request({"action": "LOG"})
        if response['status'] == 'success':
            log = response.get("log", [])
            log_text = "\n".join([f"{op['timestamp']} - {op['operation']}"
({op['id']}): {op['status']}" for op in log[-10:]])
            messagebox.showinfo("Registro de Operaciones", log_text)
        else:

```

```
        messagebox.showerror("Error", response['message'])
    except Exception as e:
        messagebox.showerror("Error", str(e))

def thread_action(self, target_func):
    """Ejecuta operaciones en hilo secundario"""
    if not self.connected:
        messagebox.showwarning("Atención", "Debe conectarse primero.")
        return

    thread = threading.Thread(target=target_func, daemon=True)
    thread.start()

if __name__ == "__main__":
    root = tk.Tk()
    app = CryptoVaultClient(root)
    root.mainloop()
```